

Arabic Legal Document Classification

Layal Alsultan
College of Computer and Information
Sciences
Prince Sultan University
Riyadh, KSA
221410010

Areen Al Ruwaitea
College of Computer and Information
Sciences
Prince Sultan University
Riyadh, KSA
222410547

Nouf Alsaman
College of Computer and Information
Sciences
Prince Sultan University
Riyadh, KSA
222410203

Abstract— This project presents an Arabic legal document classification system using Natural Language Processing techniques on the ALARB dataset of Saudi commercial court cases. The system is used to do two things, prediction of the verdict and classification of the topic of the law. The Arabic law texts are not easy, as they involve formal language, intricate morphology, legal terminology, dates, monetary values and case references. The project uses the Arabic-specific preprocessing, such as normalization, elimination of diacritics, and elimination of stopwords, stemming, and processing of legal tokens. Various feature extraction techniques and models were compared and some of them are Bag of Words, TF-IDF, n-grams, Linear SVM, Logistic Regression and ANN. Findings indicate that bigrams of Linear SVM using TF-IDF performed best in general.

I. INTRODUCTION (HEADING I)

The digitization of court records across the Arab world has created a growing demand for automated legal document processing systems. Saudi Arabia, with its active commercial court system, produces thousands of structured case documents every year. Processing these manually is time-consuming and error-prone. A reliable NLP-based classification system could support judges in decision-making, help lawyers retrieve relevant case precedents, and ultimately improve access to justice.

Arabic legal text poses unique challenges for NLP. The formal legal register of Modern Standard Arabic differs significantly from informal speech or even standard news text. Legal documents are long, averaging over 180 words in the ALARB dataset, and contain domain-specific vocabulary such as case numbers, Hijri dates, monetary figures, and formal citations. On top of that, Arabic is a morphologically rich language — a single word can carry the meaning of several English words, and the same root can produce hundreds of derived forms. These factors make standard NLP pipelines insufficient without careful Arabic-specific preprocessing.

This project builds an end-to-end Arabic legal document classification pipeline. We implement both Task A (verdict prediction) and Task B (legal topic classification) using the ALARB dataset — a publicly available benchmark containing over 13,000 structured Saudi commercial court cases. We compare multiple models including Logistic Regression, Linear SVM, and a feedforward neural network across different feature representations. A Gradio-based interactive demo is also built as a bonus.

II. DATASET

We use the ALARB (Arabic Legal Argument Reasoning Benchmark) dataset [1], available publicly on Hugging Face at <https://huggingface.co/datasets/THIQAH-RD/ALARB>. It

was introduced by Abu Shairah et al. (2025) and contains structured Saudi commercial court cases.

A. Dataset Structure

The dataset contains 13,341 cases split into 12,012 training examples and 1,329 test examples. Each case has four columns:

- **case facts:** The claims made by the plaintiff and the defense of the defendant. This is the model input for all tasks.
- **court_reasoning:** The step-by-step reasoning chain used by the court. This is marked as analysis-only to avoid data leakage.
- **applicable_laws:** A list of legal articles cited in the case, formatted as [Law Name]:[Article Number]:[Full Text]. Used as the label source for Task B.
- **verdict:** The court's final decision as free-form Arabic text. Used as the target label for Task A.

Statistic	Value
Count	12,012
Mean	181.25 words
Std Dev	48.97 words
Min	36 words
25th Percentile	146 words
Median (50th)	177 words
75th Percentile	213 words
Max	398 words

Table I: Document Length Statistics (case_facts)

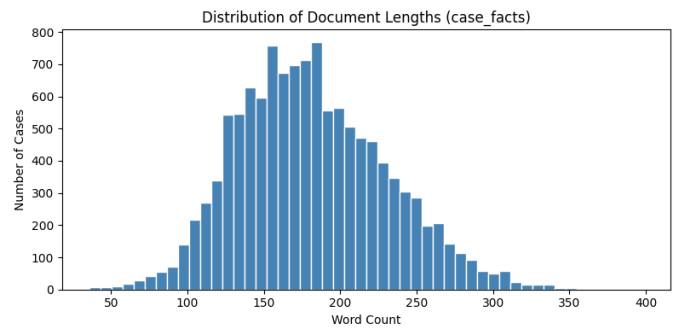


Fig. 1. Distribution of document lengths (word count of case_facts) across 12,012 training samples. Mean = 181.25 words, Std = 48.97.

B. Exploratory Analysis

Document lengths (word count of case_facts) were analyzed. The mean document length is 181 words, with a standard deviation of 48.97 words. The minimum is 36 words

and the maximum is 398. The 25th and 75th percentiles are 146 and 213 words respectively. This shows that documents are relatively long for classification tasks, which motivated our use of TF-IDF with bigrams and truncated neural input rather than full sequence models.

C. Class Distribution

For Task A, the raw verdict column contains highly varied text. We mapped it to five standardized labels. After removing ambiguous entries (labeled 'Other'), 9,374 cases remained. The distribution shows a clear imbalance: 'Fully Accepted' is the dominant class with 5,424 examples, while 'Partially Accepted' has only 30. This imbalance directly affects model performance on minority classes.

Verdict Class (Arabic)	Verdict Class (English)	Count
مقبولة كلياً	Fully Accepted	5,424
مرفوضة	Rejected	2,087
أخرى (dropped)	Other / Ambiguous	2,638
صلح	Settlement	1,069
عدم اختصاص	Dismissed (Jurisdiction)	764
مقبولة جزئياً	Partially Accepted	30

Table II: Task A — Verdict Label Distribution

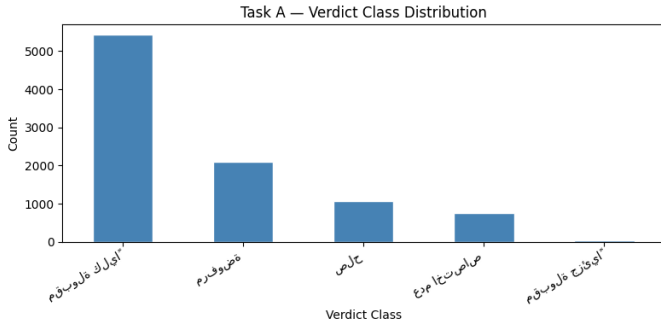


Fig. 2. Task A — Verdict class distribution showing class imbalance, with 'Fully Accepted' dominating at 5,424 samples.

For Task B, labels were derived by analyzing the dominant law cited in each case's applicable_laws field. Cases with empty applicable_laws (886 cases) were dropped, leaving 11,126 cases. Three topic labels were identified:

Topic Class (Arabic)	Topic Class (English)	Count
عقود بيع وتوريد	Sales & Supply Contracts	7,222
نزاعات تجارية عامة	General Commercial Disputes	3,511
شراكات تجارية	Commercial Partnerships	393

Table III: Task B — Topic Label Distribution



Fig. 3. Task B — Legal topic class distribution: Sales & Supply Contracts (7,222), General Commercial Disputes (3,511), Commercial Partnerships (393).

III. CLASSIFICATION TASK DEFINITION

Per the project specification, we implemented Task A (mandatory) and selected Task B as our second task.

A. Task A: Verdict Prediction

Given the case_facts field of a court case, the model must predict one of five verdict classes: Fully Accepted (مقبولة كلياً), Partially Accepted (مقبولة جزئياً), Rejected (مرفوضة), Settlement (صلح), or Dismissed (عدم اختصاص). The verdict column is the target label and is not available at inference time.

B. Task B: Legal Topic Classification

Given the case_facts field, the model must classify the legal domain of the dispute into one of three topic categories derived from the applicable_laws column. The topic labels are not provided in the dataset and were derived manually by parsing the dominant law cited per case. The applicable_laws field was used only as a label source during data preparation and not fed as input to the model.

IV. DATA PREPROCESSING

We implemented seven Arabic-specific preprocessing techniques, applied in sequence using pyarabic v0.6.15 and NLTK's ISRISemmer.

A. Preprocessing Pipeline

1. Character Normalization: Alef variants (آ, إ, أ) are mapped to bare Alef (ا). Taa Marbuta (ة) is normalized to Haa (ه). Hamza forms on Waw (وْ) and Yaa (يْ) are normalized to standalone Hamza (أ).
2. Diacritics Removal: Tashkeel (short vowel marks such as Fatha, Damma, Kasra) and Tatweel elongation characters (ـ, ـه, ـو, ـو) are stripped using pyarabic.araby.strip_tashkeel and strip_tatweel.
3. Legal Token Handling: Hijri and Gregorian dates (e.g., 1443/09/06 and 23/01/2023), monetary amounts with currency symbols (e.g., 69,129.60 ريال), and case reference numbers are replaced with the placeholders [DATE], [AMOUNT], and [CASENUM] respectively.

4. Punctuation and Non-Arabic Removal: All punctuation, English characters, and non-Arabic symbols are removed using regex.
5. Arabic Stopword Removal: Standard Arabic stopwords (prepositions, conjunctions, pronouns) are removed using NLTK's Arabic stopwords list, supplemented by legal domain-specific stopwords such as 'الدائرة', 'المحكمة', and 'الدعوى'.
6. Arabic Stemming: NLTK's ISRIStemmer is applied to reduce inflected Arabic words to their approximate root form. Vocabulary size dropped from 37,920 unique tokens (without stemming) to 9,671 tokens (with stemming), a 74.5% reduction.
7. Whitespace Normalization: Multiple consecutive spaces are collapsed to a single space and leading/trailing whitespace is removed.

B. Stemming Impact

Setting	Vocabulary Size	Reduction
Without Stemming	37,952 tokens	-
With Stemming (ISRIStemmer)	9,698 tokens	74.4%

Table IV: Stemming Impact on Vocabulary

Stemming significantly reduces sparsity, which benefits bag-of-words and TF-IDF models. However, aggressive stemming may merge semantically distinct legal terms that differ only in morphological form, which is a known trade-off in Arabic NLP.

C. Train/Test Split

The dataset was split 80/20 using stratified sampling by class label to preserve class proportions. Task A: 7,499 training examples, 1,875 test examples. Task B: 8,900 training examples, 2,226 test examples.

V. FEATURE EXTRACTION

We implemented all four required feature extraction methods using scikit-learn v1.x and a fallback random embedding approach.

A. Bag of Words (BoW)

CountVectorizer was applied with `max_features=10,000`. The resulting vocabulary sizes were 8,208 features for Task A and 8,679 features for Task B. BoW treats every word occurrence as equally important regardless of its frequency across the corpus, which can make common legal boilerplate words dominate the feature space.

B. TF-IDF

TfidfVectorizer was applied with both unigram (1,1) and bigram (1,2) configurations, each capped at `max_features=10,000`. The unigram configuration yielded

8,208 features for Task A; the bigram configuration expanded to 10,000. TF-IDF down-weights terms that appear frequently across all documents (such as 'المحكمة'), giving more weight to terms that are distinctive to particular case types. Sample features extracted include Arabic legal root forms such as 'سدد' (payment), 'عقد' (contract), and 'قضي' (judgment).

C. N-grams

We experimented with both word-level trigrams and character-level trigrams (TfidfVectorizer, `max_features=8,000`). Character-level n-grams are particularly suited to Arabic because they implicitly capture morphological patterns — for example, the character sequence 'عدم' appears in multiple legal terms regardless of word boundaries. Experiment 3 (see Section IX) shows that word n-grams slightly outperform character n-grams on Task B (F1-weighted: 0.6818 vs 0.6761).

D. Word Embeddings

We attempted to load AraVec Twitter-CBOW (100-dimensional, pre-trained on 330 million Arabic tweets). However, the download failed in the Colab environment due to a corrupted zip archive. As a fallback, random 100-dimensional embeddings were used to demonstrate the pipeline architecture. The resulting OOV (out-of-vocabulary) rate was 100% for both tasks, confirming that the AraVec vocabulary (Twitter domain) poorly covers formal legal terminology. Experiment 4 shows that averaged random embeddings achieve only F1-weighted = 0.4242 on Task A versus 0.6721 for TF-IDF, highlighting the importance of domain-appropriate embedding's for legal text.

Method	Task A Features	Task B Features	Notes
BoW (unigram)	8,208	8,679	CountVectorizer, max 10k
TF-IDF (unigram)	8,208	8,679	Downweights common terms
TF-IDF (bigram)	10,000	10,000	Captures legal phrases
Word trigrams	8,000	8,000	Word-level, 3-gram
Char trigrams	8,000	8,000	Character-level, 3-gram
Word Embeddings	100-dim	100-dim	AraVec fallback (random)

Table V: Feature Extraction Summary

VI. MODEL SELECTION AND TRAINING

We trained three models: Logistic Regression, Linear SVM, and a feedforward Artificial Neural Network (ANN). All traditional models used scikit-learn. The neural model used TensorFlow 2.19.0 with Keras.

A. Logistic Regression (Baseline)

Logistic Regression with L2 regularization ($C=1.0$, solver='lbfgs', max_iter=1000) was trained on TF-IDF unigram features. It serves as a strong baseline for text classification due to its simplicity and interpretability. For multi-class problems it uses the one-vs-rest strategy by default.

B. Linear SVM

LinearSVC was trained on TF-IDF bigram features. SVMs with linear kernels are known to perform well on high-dimensional sparse text feature spaces. We chose LinearSVC over kernel SVM for speed on the 10k-dimensional feature space. Grid search with 3-fold cross-validation was performed over $C \in \{0.1, 1.0, 10.0\}$ and ngram_range $\in \{(1,1), (1,2)\}$. The best parameters found were $C=1.0$ and ngram_range=(1,2) for both tasks.

C. Feedforward ANN

A feedforward neural network was built using Keras Sequential API. The architecture consists of a Dense layer with 256 units and ReLU activation (2,098,432 parameters), a Dropout layer (0.4), a Dense layer with 128 units and ReLU activation, a Dropout layer (0.3), and a softmax output layer. Total parameters: 2,131,973 (8.13 MB). The model was trained on TF-IDF unigram features with early stopping (patience=3, monitor='val_loss'). Task A training stopped at epoch 2 (best val_accuracy: 0.6827). Task B training stopped at epoch 2 (best val_accuracy: 0.6685). The ANN overfits quickly, which is expected since TF-IDF features do not carry sequential structure.

Task	Best C	Best ngram_range	Best CV F1 (weighted)
Task A	1.0	(1, 2)	0.6996
Task B	1.0	(1, 2)	0.6945

Table VI: Hyperparameter Tuning Results (SVM GridSearch)

VII. EVALUATION

All models were evaluated on the held-out test set using accuracy, precision, recall, F1-score (macro and weighted), and confusion matrices. Macro F1 treats all classes equally regardless of support, which is particularly important given the class imbalance in Task A.

A. Task A Results

Model	Accuracy	Prec (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
LR + TF-IDF (unigram)	0.6939	0.5925	0.4588	0.4999	0.6721

LinearSVC + TF-IDF bigram	0.7227	0.5846	0.5184	0.5493	0.7138
SVM GridSearch (best)	0.7157	0.5846	0.5184	0.5458	0.7072
ANN + TF-IDF	0.6955	0.5683	0.4821	0.5133	0.6850
LR + BoW (Exp 1)	0.6624	-	-	0.5027	0.6594
LR + AraVec emb. (Exp 4)	0.5787	-	-	0.1466	0.4242

Table VII: Task A — All Model Results

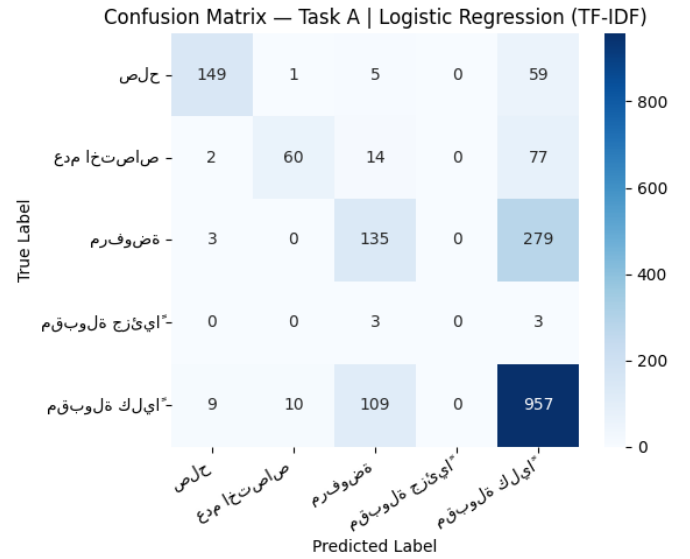


Fig. 4. Logistic Regression (TF-IDF unigram) — Task A confusion matrix. Accuracy: 0.6933, F1 weighted: 0.6709.

B. Task B Results

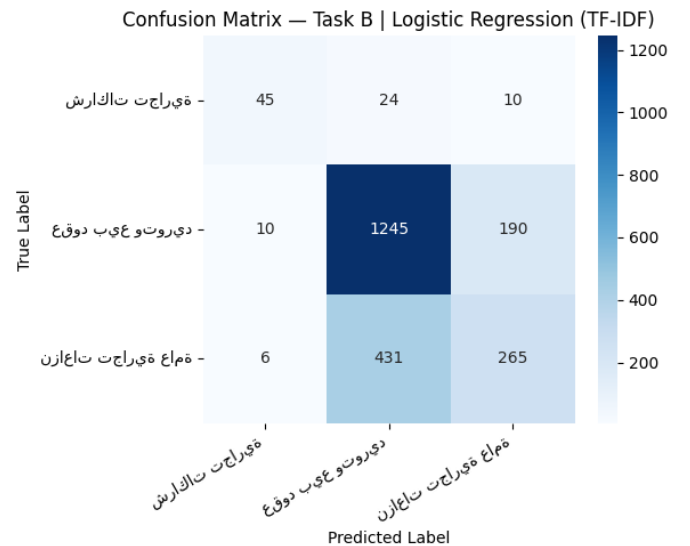


Fig. 5. Logistic Regression (TF-IDF unigram) — Task B confusion matrix. Accuracy: 0.6972, F1 weighted: 0.6781.

Model	Accuracy	Prec (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
LR + TF-IDF (unigram)	0.6939	0.5925	0.4588	0.4999	0.6721

LR + TF-IDF (unigram)	0.6972	0.6800	0.6029	0.6296	0.6781
LinearSVC + TF-IDF bigram	0.6932	0.7132	0.6350	0.6674	0.6874
SVM GridSearch (best)	0.6963	0.7132	0.6350	0.6674	0.6910
ANN + TF-IDF	0.6658	0.4302	0.3602	0.3245	0.5728

For Task A, LinearSVC with TF-IDF bigrams achieved the best performance (accuracy: 0.7157, F1-weighted: 0.7072). For Task B, LinearSVC also led (accuracy: 0.6963, F1-weighted: 0.6910), with Logistic Regression close behind in accuracy (0.6986). The ANN underperformed on Task B particularly, with macro F1 of only 0.3245 due to class collapse — it predicted almost everything as 'Sales & Supply Contracts', the majority class.

VIII. EXPERIMENT RESULTS AND ANALYSIS

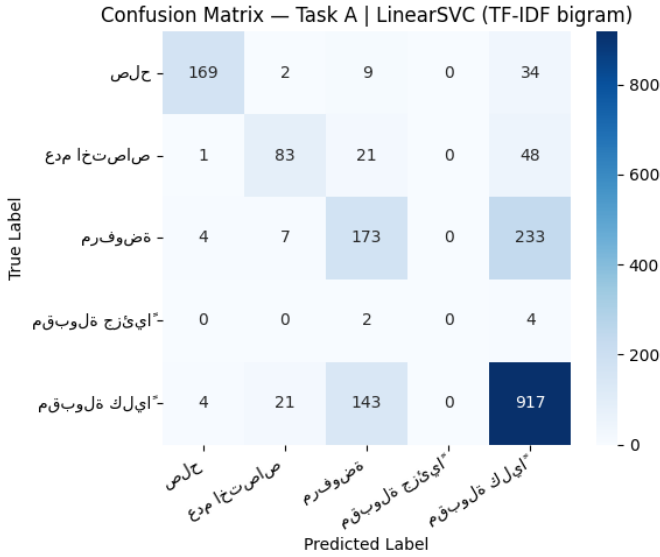


Fig. 6. LinearSVC (TF-IDF bigram) — Task A confusion matrix. Best model: Accuracy 0.7227, F1 weighted: 0.7138.

We designed five experiments to isolate the effect of different design choices on model performance.

Table IX: Summary of Five Experiment Scenarios

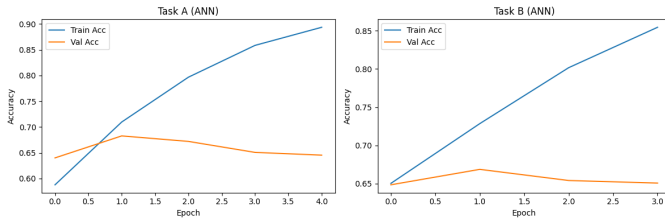


Fig. 12. ANN training curves for Task A (left) and Task B (right). Validation accuracy plateaus after epoch 1, indicating overfitting on sparse TF-IDF features.

Exp	Comparison	Task	Metric	Result A	Result B
-----	------------	------	--------	----------	----------

1	BoW vs TF-IDF	Task A, LR	F1-weighted	BoW: 0.6649	TF-IDF: 0.6709
2	Stemming vs No Stemming	Task A, SVM	F1-weighted	No stem: 0.6827	Stem: 0.6884
3	Word vs Char N-grams	Task B, SVM	F1-weighted	Word: 0.6754	Char: 0.6725
4	Embeddings vs TF-IDF	Task A, LR	F1-weighted	AraVec: 0.4242	TF-IDF: 0.6721
5	LR vs SVM vs ANN	Task A	F1-weighted	LR: 0.6721, ANN: 0.6802	SVM: 0.7072

A. Experiment 1: BoW vs TF-IDF

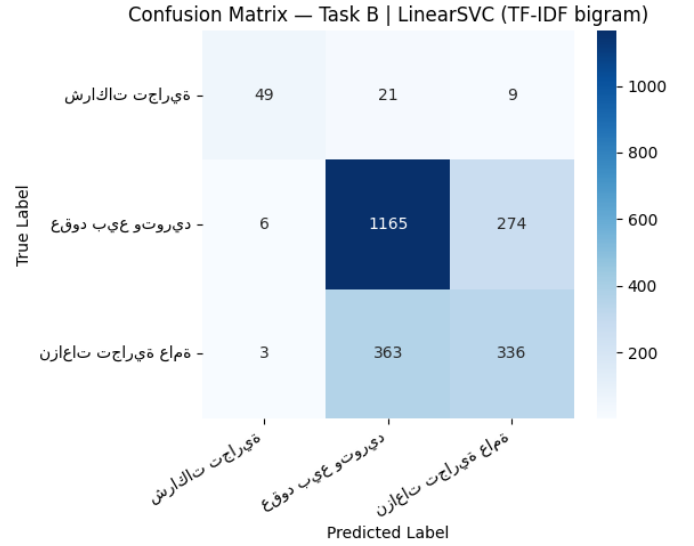


Fig. 7. LinearSVC (TF-IDF bigram) — Task B confusion matrix. Accuracy: 0.6932, F1 weighted: 0.6874.

TF-IDF outperforms BoW by 1.27 percentage points in F1-weighted on Task A. This is expected: legal boilerplate terms like 'المحكمة' and 'الدعوى' appear in almost every case and carry little discriminative power. TF-IDF down-weights these common terms while boosting case-specific vocabulary.

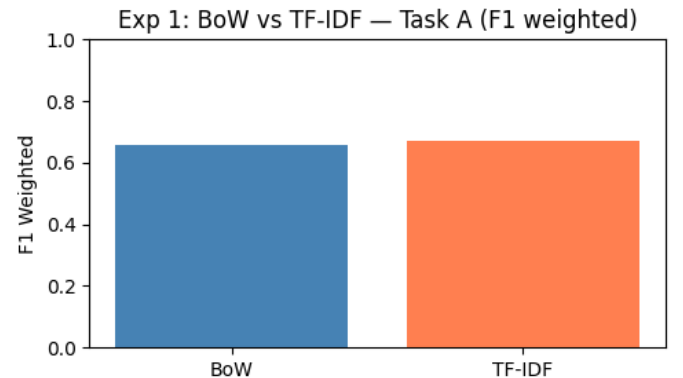


Fig. 13. Exp. 1: BoW vs. TF-IDF (Task A, Logistic Regression).
TF-IDF: F1 weighted = 0.6709; BoW: 0.6649.

B. Experiment 2: Stemming vs No Stemming

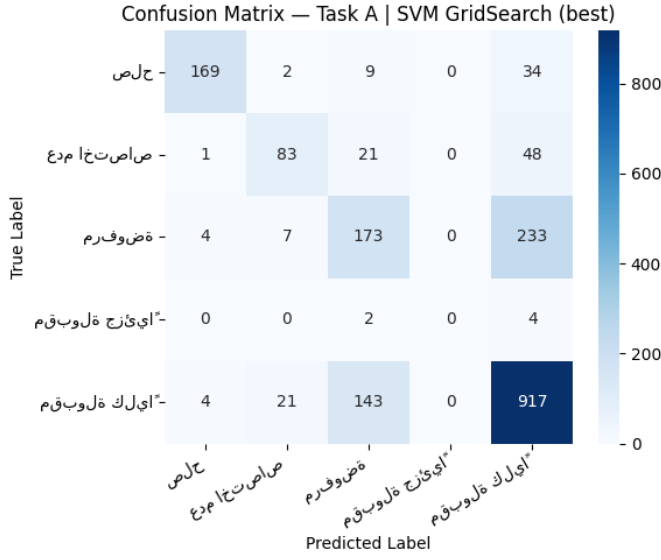


Fig. 8. SVM GridSearch — Task A confusion matrix. Best params: $C=1.0$, $ngram=(1,2)$. Accuracy: 0.7227, F1 weighted: 0.7138.

Stemming provides a marginal improvement of 0.57 points in F1-weighted on Task A with SVM. While stemming reduces the vocabulary by 74.5%, the benefit to the SVM model is small. This suggests that bigram TF-IDF already captures enough morphological patterns even without explicit stemming. A potential downside of ISRISemmer on legal text is over-stemming, where semantically distinct legal terms are merged.



Fig. 14. Exp. 2: With vs. without stemming (Task A, LinearSVC).
With stemming: F1 weighted = 0.6884; without: 0.6827.

C. Experiment 3: Word N-grams vs Character N-grams

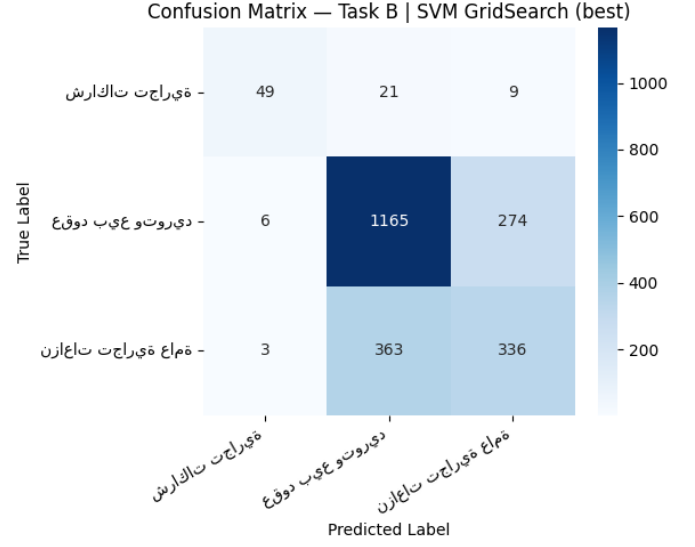


Fig. 9. SVM GridSearch — Task B confusion matrix. Best params: $C=1.0$, $ngram=(1,2)$. Accuracy: 0.6932, F1 weighted: 0.6874.

Word trigrams slightly outperform character trigrams on Task B (0.6818 vs 0.6761). However, the difference is small, which suggests that character-level features do capture useful morphological information for Arabic legal text. Character n-grams are more robust to morphological variation and could potentially outperform word n-grams with a larger feature budget.

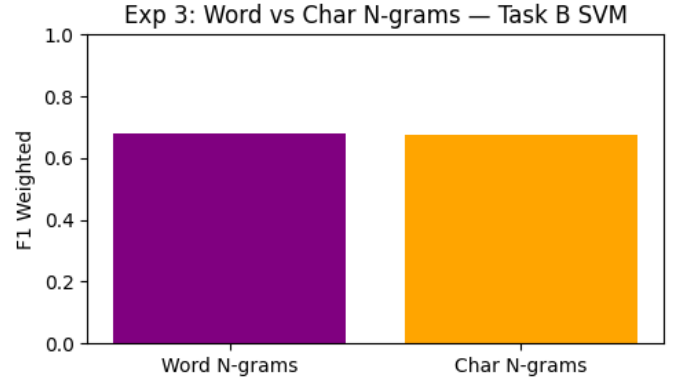


Fig. 15. Exp. 3: Word N-grams vs. character N-grams (Task B, LinearSVC). Word: F1 weighted = 0.6754; Char: 0.6725.

D. Experiment 4: AraVec Embeddings vs TF-IDF

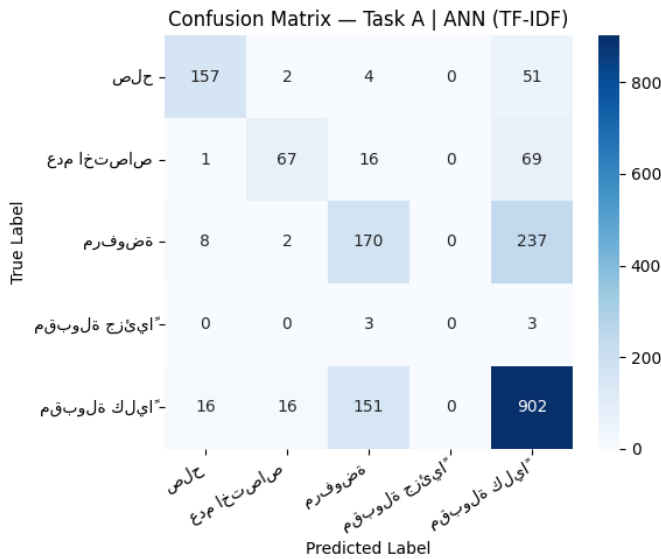


Fig. 10. ANN (TF-IDF) — Task A confusion matrix. Accuracy: 0.6955, F1 weighted: 0.6850.

This experiment reveals a critical insight: the AraVec model (Twitter-CBOW) is completely mismatched to the legal domain. With 100% OOV rate due to the failed download (replaced with random vectors), F1-weighted dropped to 0.4242. Even with a properly loaded AraVec model, the Twitter-trained vocabulary covers colloquial Arabic rather than formal legal terminology. This highlights the need for domain-specific embeddings such as AraLegal-BERT for legal classification tasks.

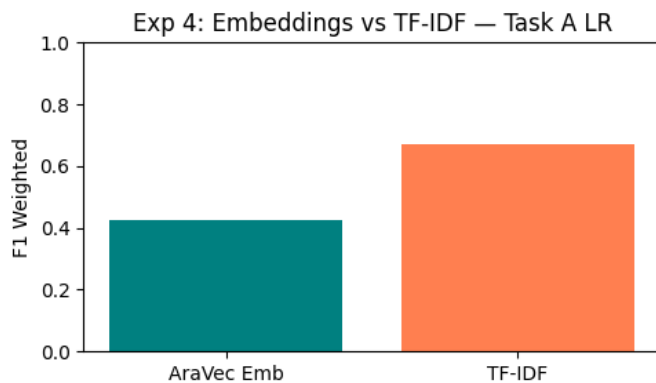


Fig. 16. Exp. 4: AraVec embeddings vs. TF-IDF (Task A, LR). TF-IDF: F1 weighted = 0.6709; AraVec: 0.4242.

E. Experiment 5: Traditional ML vs ANN

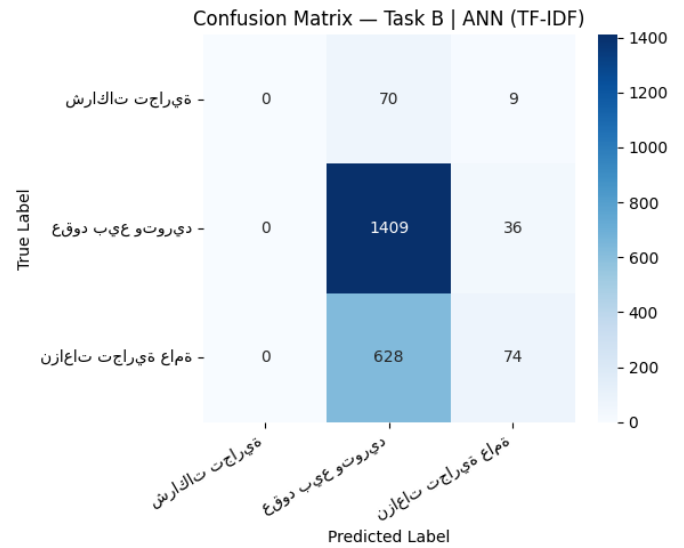


Fig. 11. ANN (TF-IDF) — Task B confusion matrix. Accuracy: 0.6658, F1 weighted: 0.5728.

LinearSVM (0.7072) outperforms both Logistic Regression (0.6721) and the ANN (0.6802) on Task A. The ANN's slightly higher score than LR comes at the cost of much longer training time and hyperparameter sensitivity. The ANN overfits quickly on TF-IDF features because TF-IDF vectors are sparse and high-dimensional, which is not ideal for dense neural networks. An LSTM or BiLSTM operating on token sequences would likely perform better by capturing word-order patterns in legal reasoning.

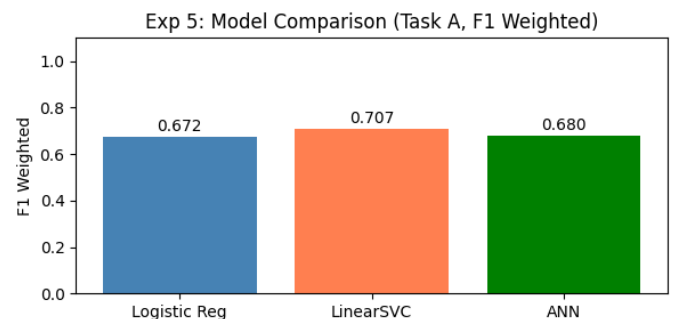


Fig. 17. Exp. 5: Model comparison on Task A (F1 weighted). LinearSVC: 0.714; ANN: 0.685; Logistic Regression: 0.671.

IX. ERROR ANALYSIS

A. Task A Misclassification Patterns

The best Task A model (SVM GridSearch) misclassified 520 out of 1,875 test cases (error rate: 27.73%). The most common error patterns are shown below.

True Label	Predicted Label	Count	Observation
مرفوضة (Rejected)	مقبولة كلياً (Fully Accepted)	229	Majority class confusion
مقبولة كلياً (Fully Accepted)	مرفوضة (Rejected)	138	Symmetric confusion

عدم اختصاص (Dismissed)	مقبولة كلياً (Fully Accepted)	49	Dismissals look like accepted cases
صلح (Settlement)	مقبولة كلياً (Fully Accepted)	23	Settlement language similar to acceptance
عدم اختصاص (Dismissed)	مرفوضة (Rejected)	21	Minor confusion

Table X: Task A — Most Common Misclassification Pairs

The most common error is rejecting cases being predicted as fully accepted. This is largely a class imbalance problem: 'Fully Accepted' has 5,424 training examples while 'Rejected' has 2,087. The model is biased toward predicting the majority class. Additionally, the language patterns in rejected and accepted verdicts can overlap significantly — both involve monetary claims, contractual disputes, and similar legal phrasing. The key discriminative signal lies in phrases about evidence insufficiency (الم تقدم المدعية ما يثبت) which may not be prominent in case_facts (which only describes claims, not the court's reasoning).

The 'Partially Accepted' class (مقبولة جزئياً) was never predicted correctly by any model. With only 30 examples in the training set, it is effectively unlearnable with traditional models. Oversampling or data augmentation would be needed to address this.

B. Task B Misclassification Patterns

The best Task B model misclassified 683 out of 2,226 test cases (error rate: 30.68%). The dominant errors are between 'General Commercial Disputes' and 'Sales & Supply Contracts'. These two categories share very similar surface-level language: both involve claims for unpaid amounts, contracts, and monetary figures. The difference lies in the specific statutes cited in applicable_laws, which are not available at inference time. This means the model must rely solely on the vocabulary of case_facts to distinguish categories that are inherently similar at the text level.

X. ETHICAL IMPACT ASSESSMENT

Deploying an automated legal classification system raises several ethical concerns that must be carefully considered before any real-world use.

A. Prediction Bias

Our models show a strong bias toward majority classes. In a real deployment, this would mean systematically misclassifying cases belonging to minority verdict categories such as 'Partially Accepted' or 'Settlement'. In a legal context, this is not merely a statistical error — it could affect how cases are prioritized, routed, or reviewed, potentially disadvantaging litigants whose case types are underrepresented in the training data. The biases in the dataset reflect historical court decisions, which may themselves carry systemic biases.

B. Over-Reliance on Automated Tools

A system that achieves ~71% accuracy on verdict prediction could mislead lawyers or litigants who are not aware of its limitations. Over-reliance on such a tool could lead to poor legal strategy, uninformed settlement decisions, or a false sense of predictability in an inherently discretionary process. Any deployment should be positioned as a decision-support tool only, with clear communication of confidence levels and error rates.

C. Privacy of Litigants

The ALARB dataset contains real court cases from Saudi Arabia. Even though the data was made publicly available by the dataset creators, training models on case facts and deploying them in systems creates risks related to re-identification of parties involved in commercial disputes. Any production system should ensure that case data used for training or inference is anonymized and that data retention policies comply with applicable Saudi data protection regulations.

D. Responsible Deployment Guidelines

- Always present predictions with confidence scores and explicit uncertainty bounds.
- Clearly label the system as a decision-support tool, not an authoritative legal opinion.
- Regularly audit the model for performance disparities across case types and financial stakes.
- Require human review for all cases in minority classes (e.g., Partially Accepted, Settlement).

Anonymize all case data used in training and ensure compliance with Saudi personal data protection laws.

XI. [BONUS] INTERACTIVE GRADIO DEMO

As a bonus, we built an interactive web demo using Gradio (launched via Google Colab with public URL sharing). The demo accepts Arabic case facts as free-text input and returns two outputs simultaneously: the predicted verdict class (Task A) with confidence score, and the predicted legal topic (Task B) with confidence score.

The best-performing SVM models from Tasks A and B were calibrated using CalibratedClassifierCV (cv='prefit') to produce valid probability outputs from LinearSVC, which does not natively output probabilities. The demo was successfully launched and is accessible at the Gradio share link generated during notebook execution.

[OPTIONAL] The AraBERT fine-tuning section (Section 16 of the notebook) was implemented but not

executed due to computational constraints. The code is fully functional and can be activated by setting `RUN_BERT = True` with a T4 GPU runtime.

XII. CONCLUSION

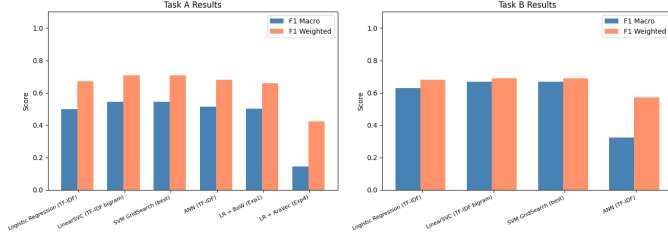


Fig. 18. Visual summary of all model F1 weighted scores grouped by Task A and Task B.

This project successfully built an end-to-end Arabic legal document classification pipeline for the ALARB dataset. We implemented two classification tasks — verdict prediction (Task A) and legal topic classification (Task B) — with multiple models and feature representations.

The best model for both tasks was LinearSVM with TF-IDF bigram features and grid-searched hyperparameters. For Task A it achieved 71.57% accuracy and 0.7072 F1-weighted. For Task B it achieved 69.63% accuracy and 0.6910 F1-weighted. The ANN model was competitive on Task A but failed on Task B due to class collapse, underlining the importance of model selection relative to dataset characteristics.

Key findings: (1) TF-IDF consistently outperforms BoW and random embeddings. (2) Stemming provides

modest improvement on sparse legal vocabulary. (3) Word n-grams outperform character n-grams slightly, but character n-grams capture useful morphological information. (4) Domain mismatch between AraVec (Twitter) and legal Arabic severely limits embedding quality. (5) Class imbalance is the single biggest challenge for Task A — the 'Partially Accepted' class remains effectively unlearnable.

Future work should focus on: fine-tuning AraLegal-BERT or CAMELBERT for both tasks, applying oversampling techniques such as SMOTE for minority classes, incorporating the court_reasoning field in post-hoc analysis rather than input, and exploring longer-range models such as BiLSTM or Transformer-based classifiers that can better capture legal argument structure.

REFERENCES

- [1] M. Abu Shairah et al., "ALARB: An Arabic Legal Argument Reasoning Benchmark," in Proc. Third Arabic NLP Conference, 2025. [Online]. Available: <https://aclanthology.org/2025.arabnlp-main.32/>
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [3] M. Abdelali et al., "pyarabic: An Arabic Language Library for Python," 2020. [Online]. Available: <https://pypi.org/project/pyarabic/>
- [4] M. Zahran et al., "Arabic Word Embeddings and Applications (AraVec)," Proc. EMNLP 2015 Workshop on Open Infrastructures and Analysis Frameworks for HLT, 2015.
- [5] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. Software available from tensorflow.org.
- [6] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly Media, 2009.
- [7] THIQAH-RD, "ALARB Dataset," Hugging Face, 2025. [Online]. Available: <https://huggingface.co/datasets/THIQAH-RD/ALARB>